

Soutenance de Stage de Fin d' Études

Spécialité: Sécurité des systèmes d'informations et des infrastructures

Mise en place d'une solution WAF open source

Réalisé par: **May BEJAOU**



Encadrant académique: **Madame Nihel BEN YOUSSEF**

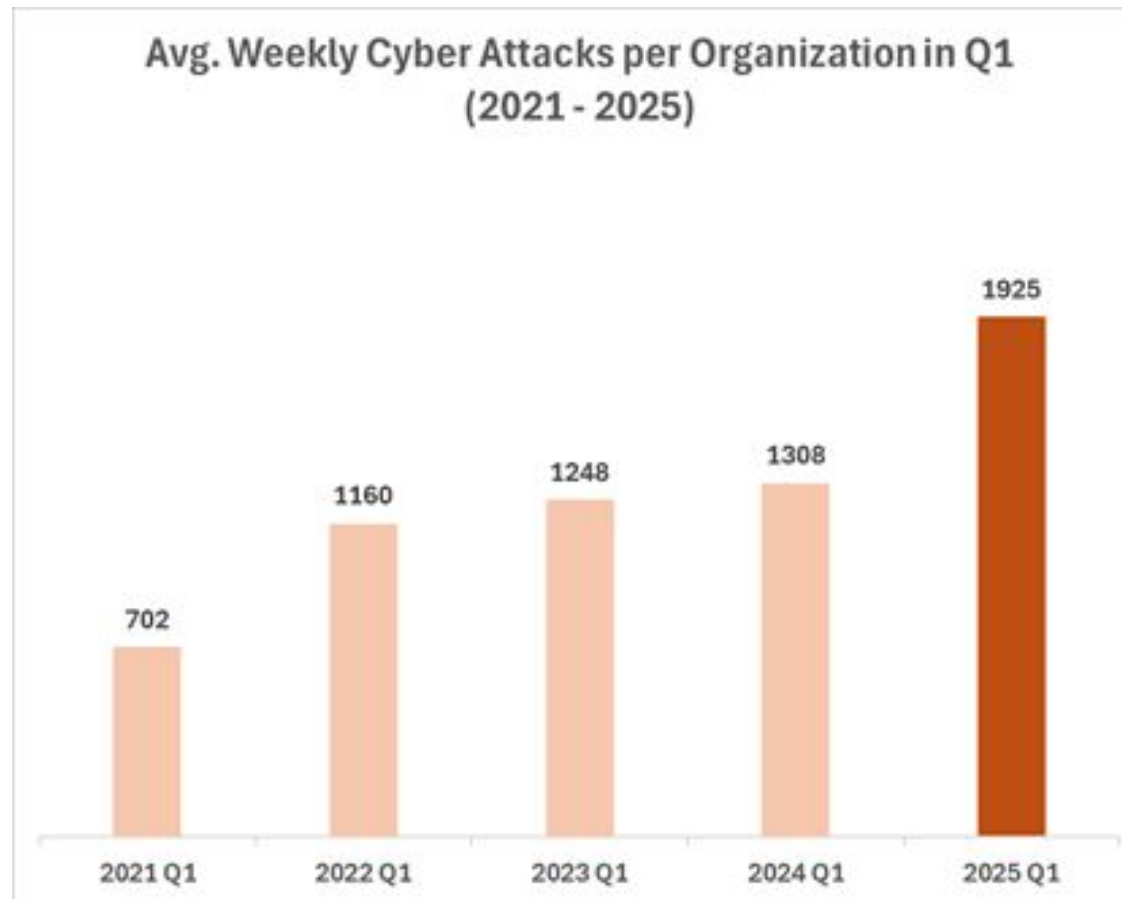
Encadrant professionnel: **Monsieur Chawki RHIMI**

2024-2025

Contexte et problématique:

Motivation

À l'échelle mondiale, les entreprises ont subi **en moyenne 1 925 tentatives d'intrusion par semaine**, soit une **hausse de plus de 38 %** par rapport à l'année précédente.



Contexte et problématique:

Organisme d'accueil

Le groupe **Wevioo** accompagne ses clients dans leurs projets de transformation digitale en leur apportant son expertise dans le Consulting, le Digital et l'Innovation.



Filiale du Groupe WEVIOO, **ADACTIM** est un Opérateur De Services Managés opérant à l'international via une présence en Europe, au Maghreb et en Afrique. Elle met à la disposition de ses partenaires des solutions innovantes et leur permet l'externalisation de l'exploitation des infrastructures SI & Cloud



Contexte et problématique:

Organisme d'accueil



**27 années
croissance**



**460
Employees**



**500
Projets**



**05
Bureaux**

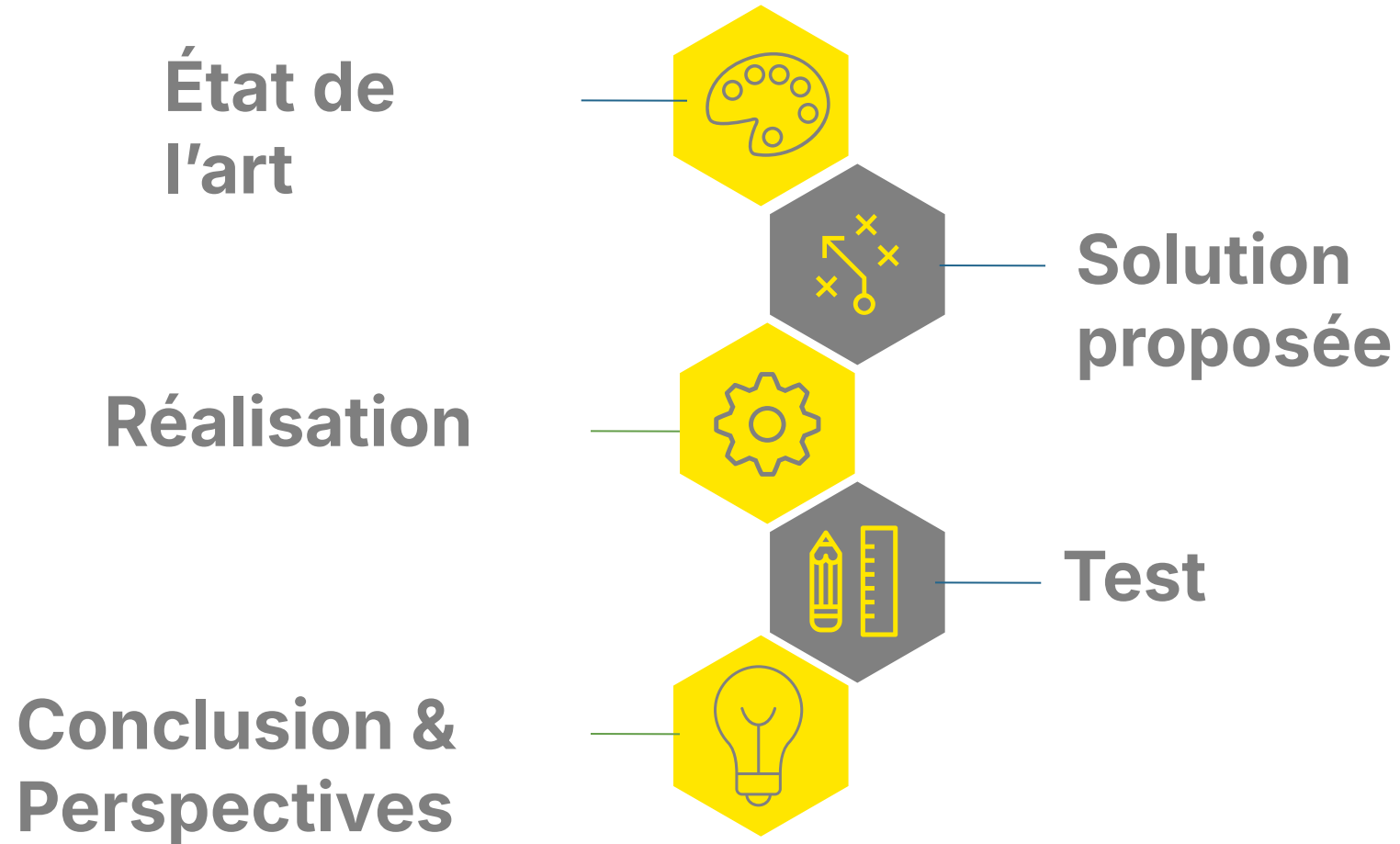


| Problématique

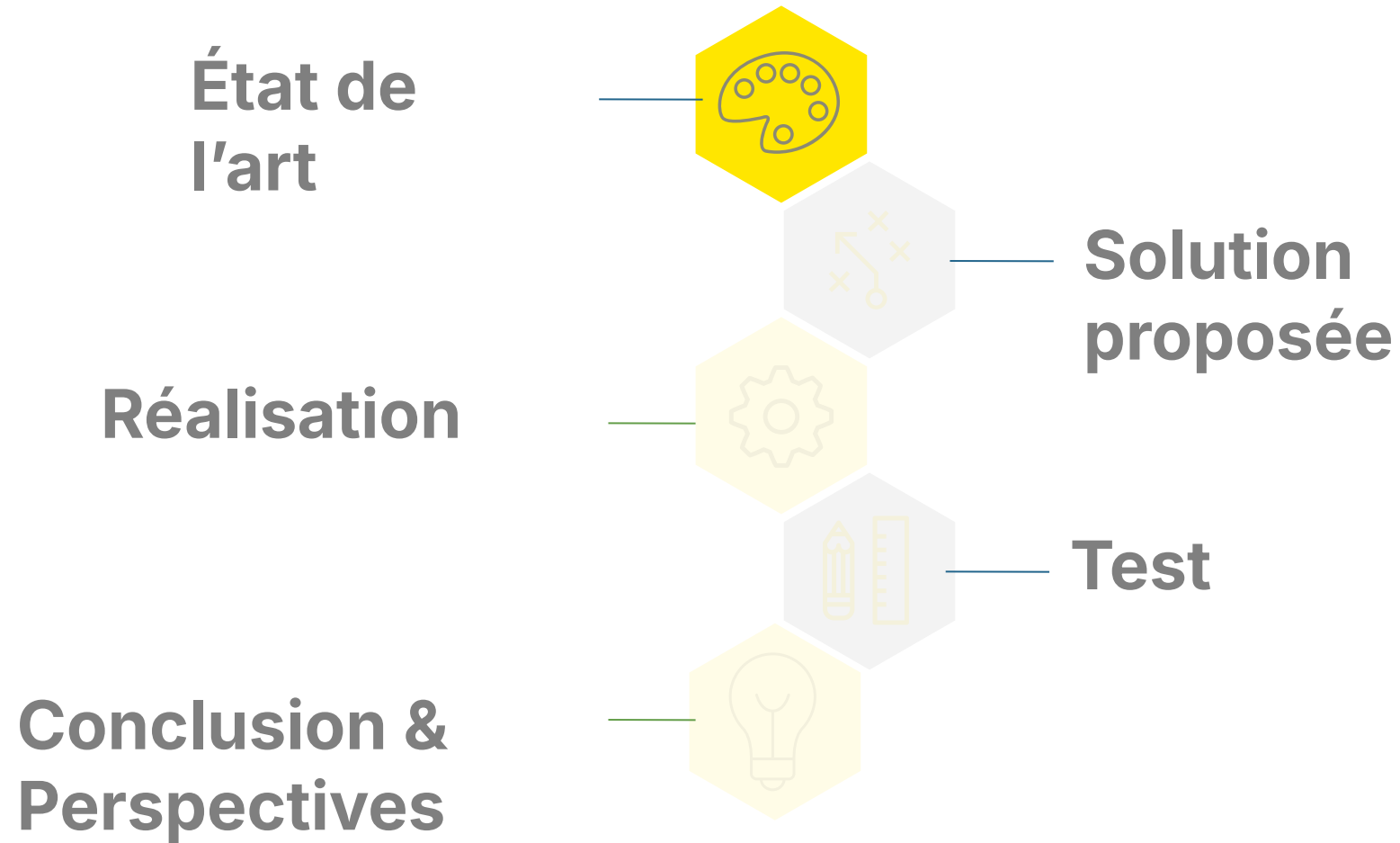
- Vulnérabilité accrue aux attaques web avancées
- Absence d'inspection approfondie des requêtes HTTP/HTTPS
- Non-conformité réglementaire
- Posture de sécurité affaiblie



Plan



Plan



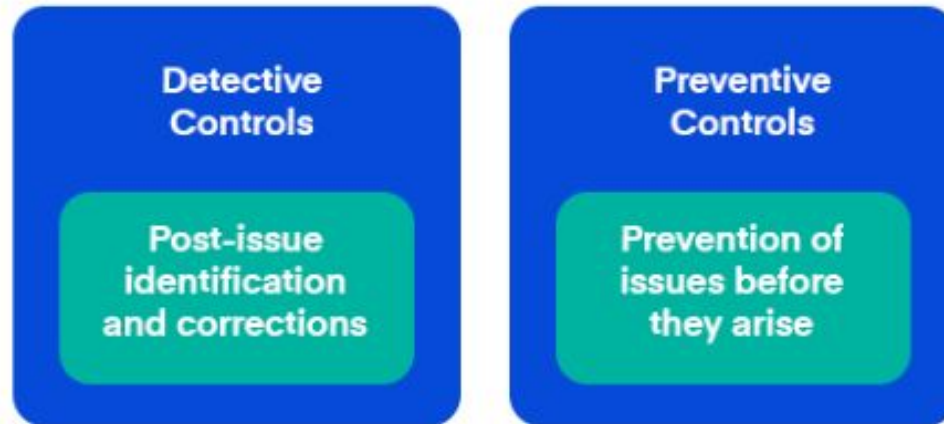
État de l'art:

Méthodes de prévention

Il existe deux principaux types de méthodes de prévention:

- Approche détective
- Approche préventive

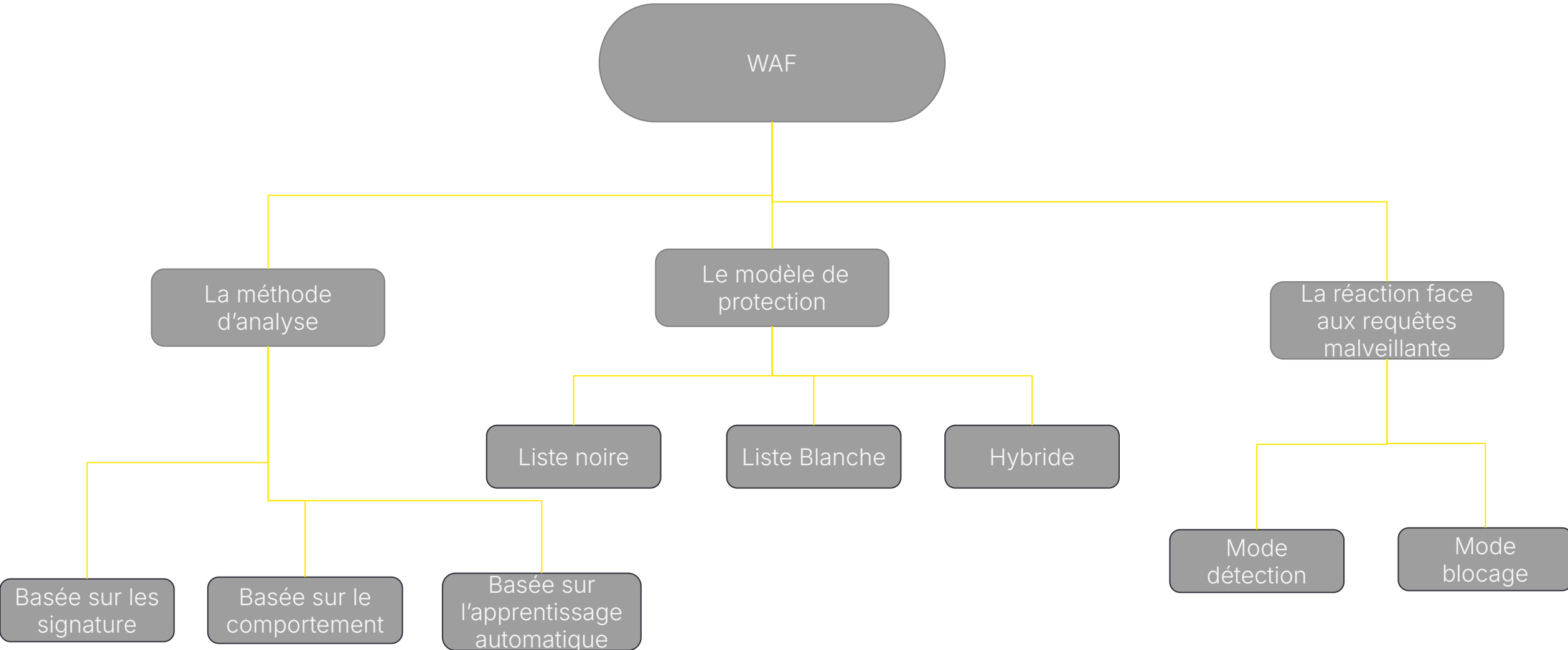
Detective vs. Preventive Controls: A Balanced Approach



État de l'art:

WAF

Les WAFs sont organisées comme illustré :



État de l'art:

Etude comparative

Il existe solutions WAF sur le marché, mais chacun d'entre eux présente certaines faiblesses.

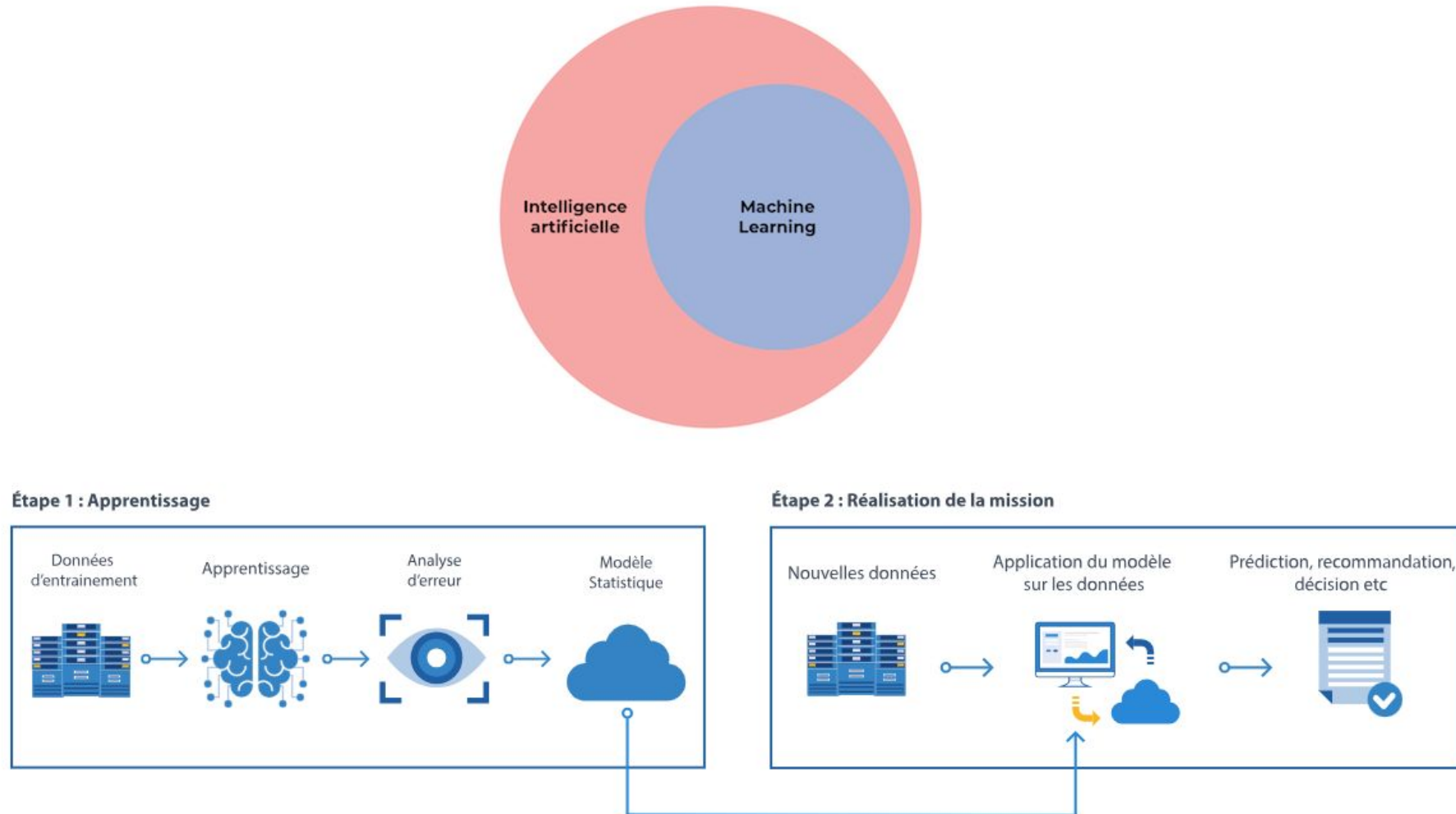
	BunkerWeb	Safeline	Naxi	Open AppSec
La méthode d'analyse	Basé sur des signatures OWASP CRS	Basé sur des signatures personnalisées	Basé sur des signatures	Basé sur L'apprentissage Automatique
Le modèle de protection	- Liste noire - Liste blanche	Liste noire	- Liste noire - Liste blanche	- Liste noire - Liste blanche
Protection contre les bots malveillants	✓	✓	✗	✗
Apprentissage automatique	✗	✗	✗	✓



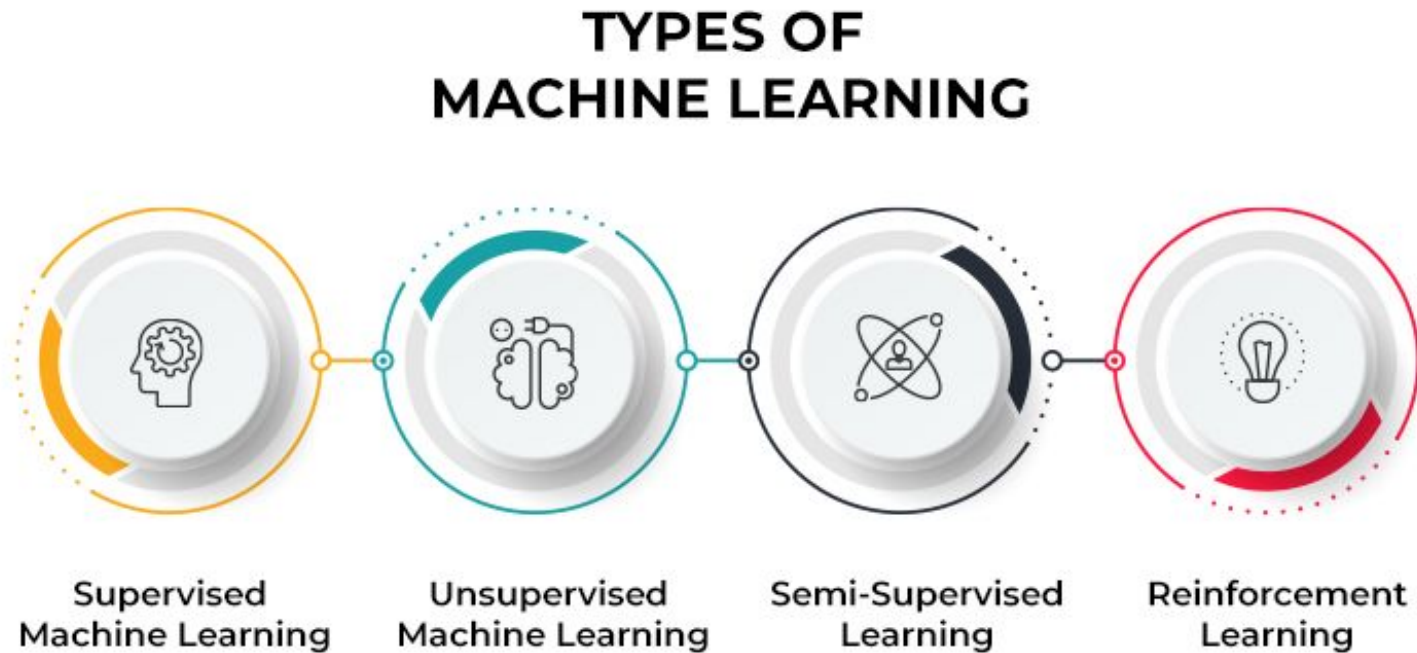
État de l'art:

Apprentissage Automatique

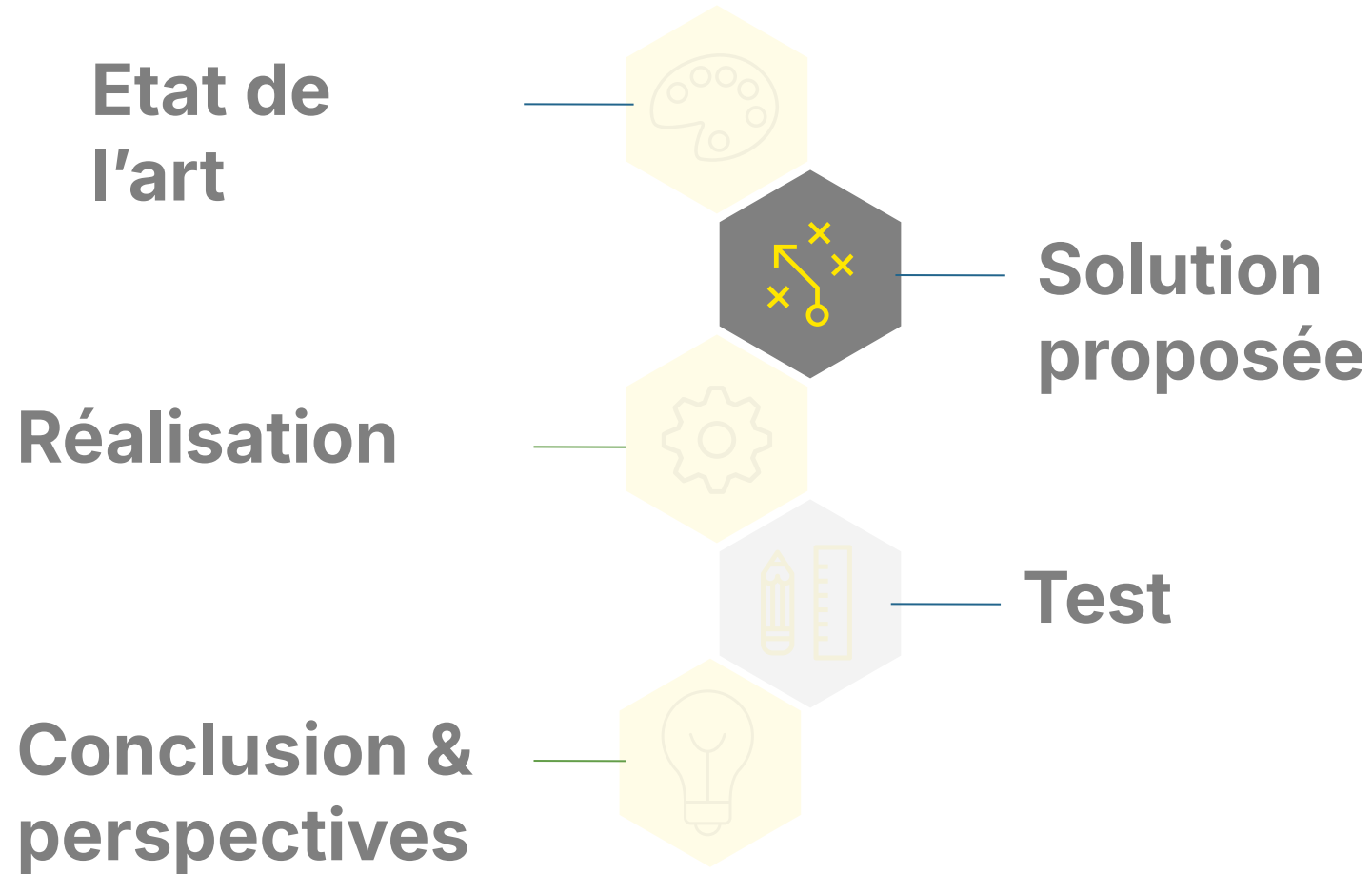
D'une manière générale, les algorithmes d'apprentissage automatique se décomposent en deux étapes



Nous identifions quatre classes courantes d'apprentissage automatique.



Plan



Solution Proposée:

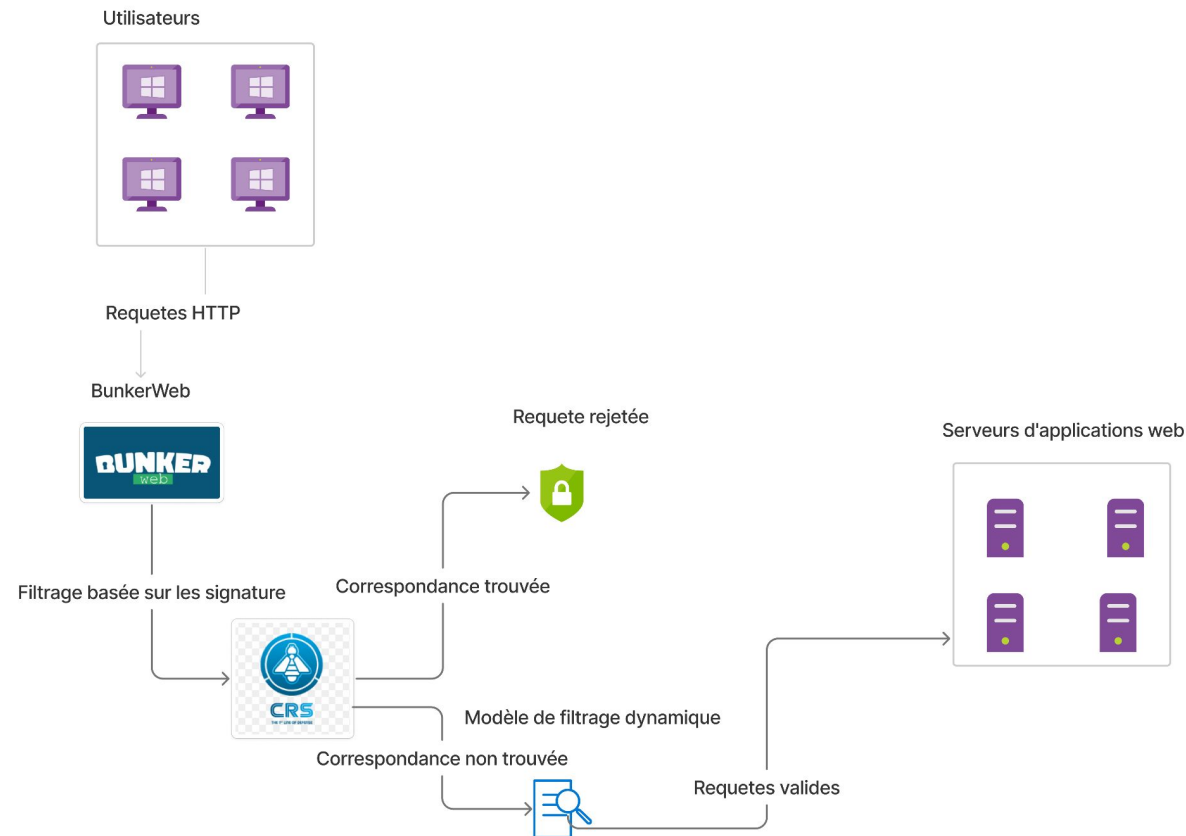
Pourquoi BunkerWeb?

- Intégration facile aux environnements existants.
- Hautement personnalisable.
- Système de plugins.
- Interface utilisateur Web exceptionnelle.
- Meilleure conformité (RGPD)

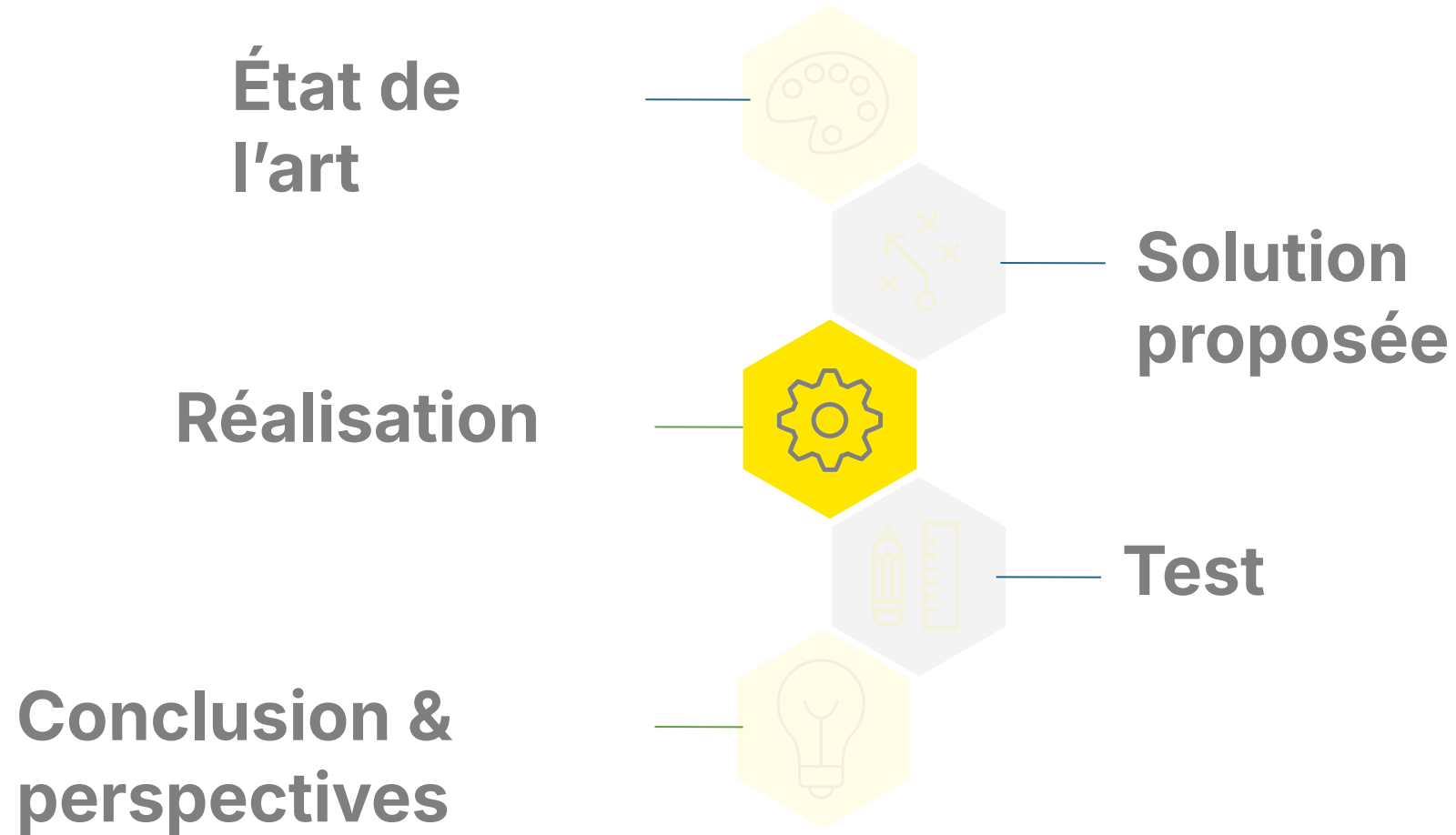


Solution Proposée

Notre solution propose le déploiement de BunkerWeb, renforcé par un module d'intelligence artificielle capable de détecter les menaces inconnues, faisant de ce WAF une solution hybride alliant règles classiques et détection intelligente.



Plan



Réalisation:

docker-compose

```
GNU nano 6.2                docker-compose.yml *
x-bw-env: &bw-env
# on utilise ancre pour permettre d'insérer cette configuration dans plusieurs
API_WHITELIST_IP: "127.0.0.0/8 10.20.30.0/24"
BUNKERWEB_ALLOWLIST: "10.0.2.15"
DATABASE_URI: "mariadb+pymysql://bunkerweb:bunkerweb@bw-db:3306/db"

services:
  bunkerweb:
    # This is the name that will be used to identify the instance in the Scheduler
    image: bunkerity/bunkerweb:1.6.0 #image Docker officielle
    ports:
      - "80:8080/tcp" # redirige le port 8080 du conteneur vers 80 du hôte
      - "443:8443/tcp"
      - "443:8443/udp" # For QUIC / protocole sécurisé basé sur UDP
    environment:
      <<: *bw-env # We use the anchor to avoid repeating the same settings for
    restart: "unless-stopped"
    networks:
      - bw-universe
      - bw-services

  bw-scheduler: #gestionnaire des tâches de bunkerweb
    image: bunkerity/bunkerweb-scheduler:1.6.0
    environment:
      <<: *bw-env
      BUNKERWEB_INSTANCES: "bunkerweb" # Définit l'instance de bunkerweb à gérer
      SERVER_NAME: ""
      MULTISITE: "yes"
      UI_HOST: "http://bw-ui:7000" # Change it if needed
    volumes:
      - bw-data:/data # This is used to persist the cache and other data like the
    restart: "unless-stopped"
    networks:
```

```
networks:
  - bw-universe
  - bw-db

bw-ui:
  image: bunkerity/bunkerweb-ui:1.6.0
  environment:
    <<: *bw-env
  restart: "unless-stopped"
  networks:
    - bw-universe
    - bw-db

bw-db:
  image: mariadb:11
  environment:
    MYSQL_RANDOM_ROOT_PASSWORD: "yes"
    MYSQL_DATABASE: "db"
    MYSQL_USER: "bunkerweb"
    MYSQL_PASSWORD: "bunkerweb"
  volumes:
    - bw-db:/var/lib/mysql
  restart: "unless-stopped"
  networks:
    - bw-db

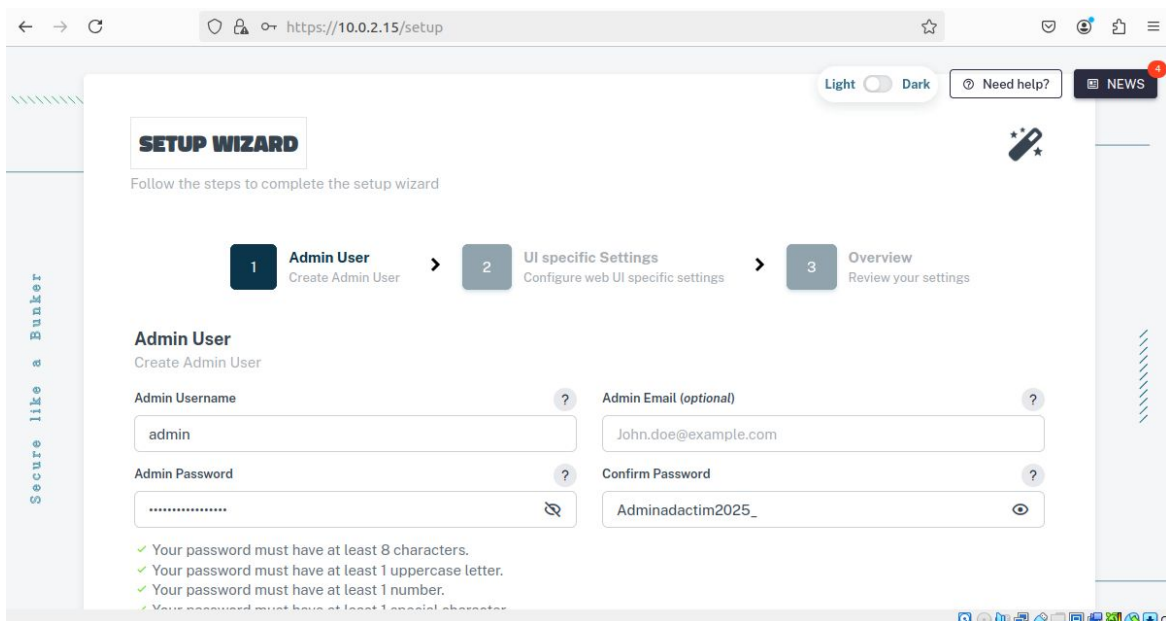
volumes: #utilisés pour stocker les données persistantes
  bw-data:
  bw-db:
```

docker-compose.yml

Réalisation:

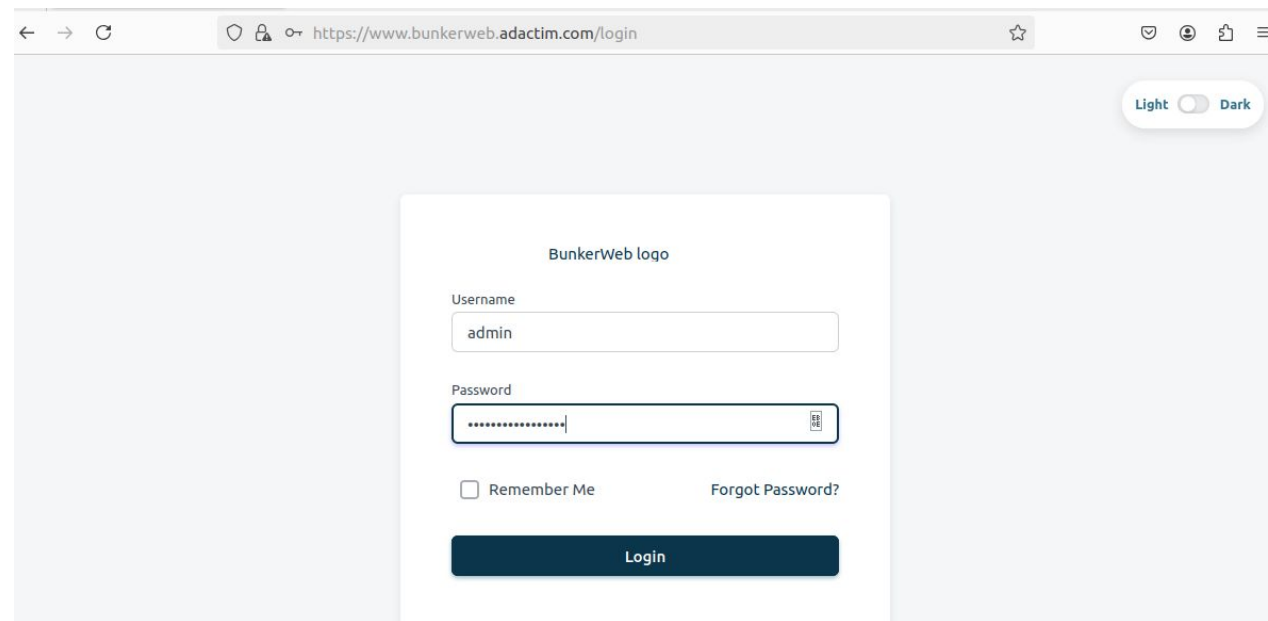
Interface utilisateur

L'interface web est une application web qui nous permet de gérer notre instance de BunkerWeb via une interface conviviale, au lieu d'utiliser la ligne de commande.



The screenshot shows the 'SETUP WIZARD' interface in a web browser at the URL `https://10.0.2.15/setup`. The interface is titled 'SETUP WIZARD' and instructs the user to 'Follow the steps to complete the setup wizard'. It features a progress bar with three steps: 1. Admin User (Create Admin User), 2. UI specific Settings (Configure web UI specific settings), and 3. Overview (Review your settings). The first step is active. Below the progress bar, the 'Admin User' section is displayed, with the subtext 'Create Admin User'. It contains four input fields: 'Admin Username' (with the value 'admin'), 'Admin Email (optional)' (with the value 'John.doe@example.com'), 'Admin Password' (masked with dots), and 'Confirm Password' (with the value 'Adminadactim2025_'). There are also checkboxes for password requirements: 'Your password must have at least 8 characters.', 'Your password must have at least 1 uppercase letter.', 'Your password must have at least 1 number.', and 'Your password must have at least 1 special character.' The interface includes a 'Light' and 'Dark' theme toggle, a 'Need help?' button, and a 'NEWS' button. A vertical sidebar on the left contains the text 'Secure like a Bunker'.

Configuration de l'interface

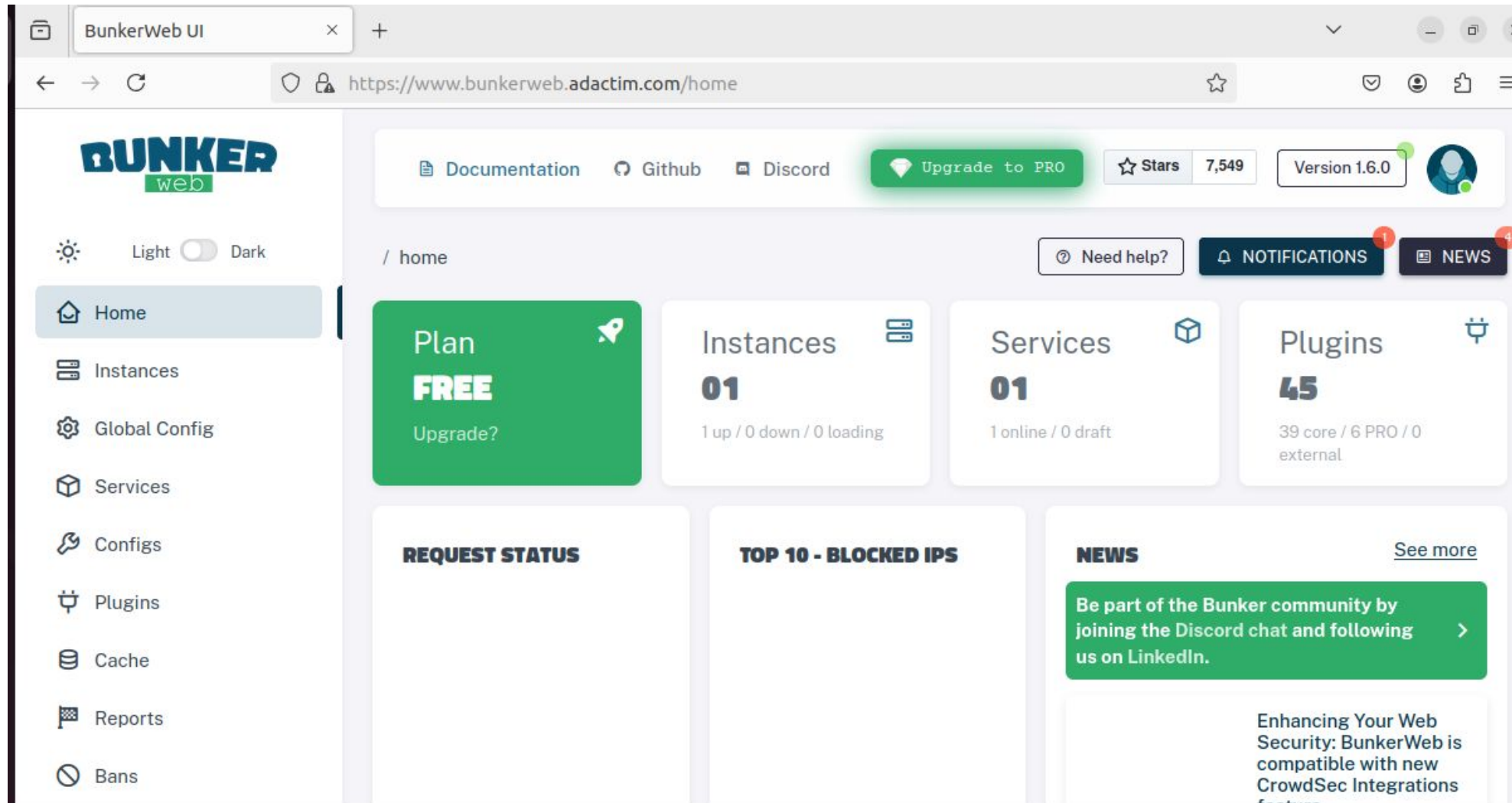


The screenshot shows the login interface in a web browser at the URL `https://www.bunkerweb.adactim.com/login`. The interface is titled 'BunkerWeb logo' and contains a 'Username' input field with the value 'admin', a 'Password' input field (masked with dots), and a 'Remember Me' checkbox. There is also a 'Forgot Password?' link and a 'Login' button. The interface includes a 'Light' and 'Dark' theme toggle.

Page d'authentification

Réalisation:

Dashboard



Réalisation: Fonctionnalités

service - Upstream server

figure the upstream server to be protected by BunkerWeb

3

HTTP - General

Configure the settings related to the HTTP(S) protocol

HTTP - General

Configure the settings related to the HTTP(S) protocol

Allowed methods

MAXIMUM BODY SIZE

HTTP2

HTTP3

HTTPS protocols

ALLOWED_METHODS

MAX_CLIENT_SIZE

HTTP2

HTTP3

SSL_PROTOCOLS

GET|POST|HEAD|OPTIONS|PUT|D

100m

HTTP2

HTTP3

TLSv1.2 TLSv1.3

< Previous

Next >

HTTP General

Templates

Reset

Online

Save

Security - Limiting

Configure the settings related to limiting requests and connections.

Activate limit connections

Maximum number of HTTP/1.X connections

Maximum number of HTTP/2 streams

Maximum number of HTTP/3 streams

Limit request Rate

Activate limit requests

Limit request URL

USE_LIMIT_CONN

LIMIT_CONN_MAX_HTTP1

LIMIT_CONN_MAX_HTTP2

LIMIT_CONN_MAX_HTTP3

LIMIT_REQ_RATE

USE_LIMIT_REQ

LIMIT_REQ_URL

25

200

200

5r/s

/

< Previous

Next >

HTTP - Headers

Configure the settings related to the HTTP headers

5

Security - Bad behavior

Configure the settings related to the automatic ban when a bad behavior is detected.

Security - Bad behavior

Configure the settings related to the automatic ban when a bad behavior is detected.

Activate bad behavior

Bad status codes

Ban duration (in seconds)

Threshold

Period (in seconds)

USE_BAD_BEHAVIOR

BAD_BEHAVIOR_STATUS_CODES

BAD_BEHAVIOR_BAN_TIME

BAD_BEHAVIOR_THRESHOLD

BAD_BEHAVIOR_COUNT_TIME

400 401 403 404 405 429 444

3600

30

60

< Previous

Next >

Mauvais comportement

Security - Blacklisting

Configure the settings related to the external blacklists.

Activate blacklisting

Activate DNSBL

Blacklist IP/network

Blacklist reverse DNS

Blacklist ASN

Blacklist User-Agent

Blacklist URI

Blacklist IP/network URLs

Blacklist reverse DNS URLs

Blacklist ASN URLs

Blacklist User-Agent URLs

Blacklist URI URLs

USE_BLACKLIST

USE_DNSBL

BLACKLIST_IP

BLACKLIST_RDNS

BLACKLIST_ASN

BLACKLIST_USER_AGENT

BLACKLIST_URI

BLACKLIST_IP_URLS

BLACKLIST_RDNS_URLS

BLACKLIST_ASN_URLS

BLACKLIST_USER_AGENT_URLS

BLACKLIST_URI_URLS

.shodan.io.censys.id

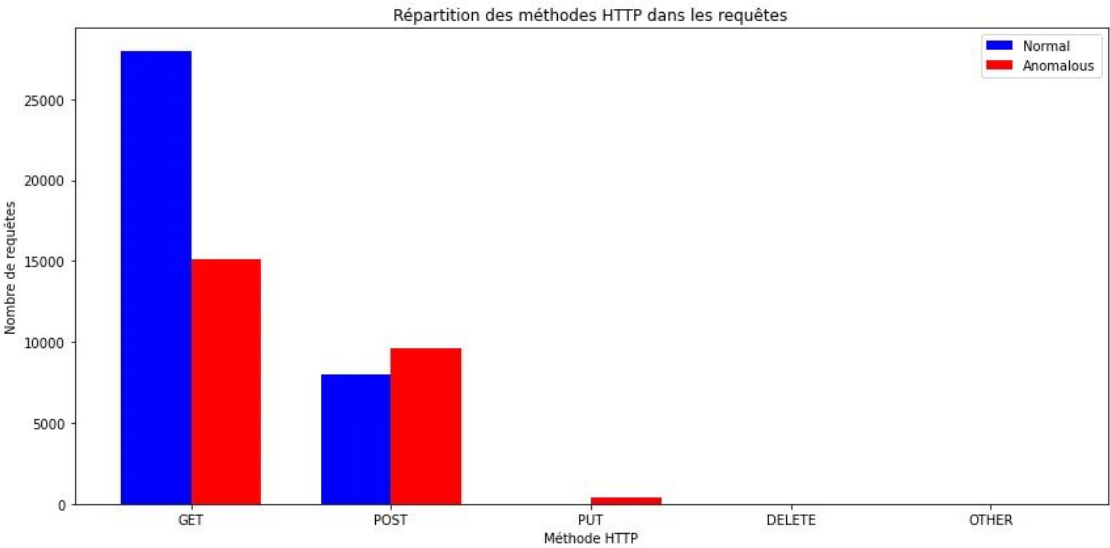
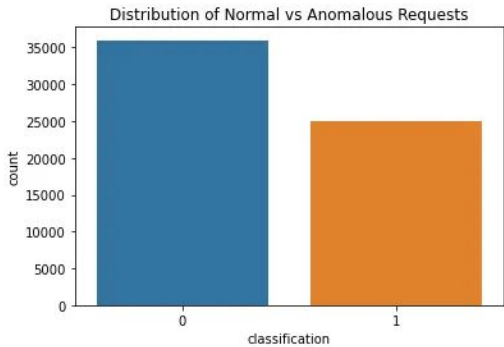
https://www.dan.me.uk/torlist/?exi

Réalisation:

Apprentissage automatique

```
Entrée [37]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(data=df, x='classification') |
plt.title("Distribution of Normal vs Anomalous Requests")
plt.show()
```



Entrée [47]: `df[df['classification'] == 1].head(10)`

Out[47]:

	Unnamed: 0	Method	User-Agent	Pragma	Cache-Control	Accept	Accept-encoding	Accept-charset	language	host
36000	Anomalous	GET	Mozilla/5.0 (compatible; Konqueror/3.5; Linux)...	no-cache	no-cache	text/xml,application/xml,application/xhtml+xml...	x-gzip, x-deflate, gzip, deflate	utf-8, utf-8;q=0.5, *,q=0.5	en	localhost:8080 JSESSION
36001	Anomalous	POST	Mozilla/5.0 (compatible; Konqueror/3.5; Linux)...	no-cache	no-cache	text/xml,application/xml,application/xhtml+xml...	x-gzip, x-deflate, gzip, deflate	utf-8, utf-8;q=0.5, *,q=0.5	en	localhost:8080 JSESSIONID
36002	Anomalous	GET	Mozilla/5.0 (compatible; Konqueror/3.5; Linux)...	no-cache	no-cache	text/xml,application/xml,application/xhtml+xml...	x-gzip, x-deflate, gzip, deflate	utf-8, utf-8;q=0.5, *,q=0.5	en	localhost:8080 JSESSIONID
36003	Anomalous	POST	Mozilla/5.0 (compatible; Konqueror/3.5; Linux)...	no-cache	no-cache	text/xml,application/xml,application/xhtml+xml...	x-gzip, x-deflate, gzip, deflate	utf-8, utf-8;q=0.5, *,q=0.5	en	localhost:8080 JSESSION
36004	Anomalous	GET	Mozilla/5.0 (compatible; Konqueror/3.5; Linux)...	no-cache	no-cache	text/xml,application/xml,application/xhtml+xml...	x-gzip, x-deflate, gzip, deflate	utf-8, utf-8;q=0.5, *,q=0.5	en	localhost:8080 JSESSION

Nombre de bonnes requêtes: 36000
Nombre de mauvaises requêtes: 25065
Méthodes normales: {'GET': 28000, 'POST': 8000, 'PUT': 0, 'DELETE': 0, 'OTHER': 0}
Méthodes anomalies: {'GET': 15088, 'POST': 9580, 'PUT': 397, 'DELETE': 0, 'OTHER': 0}

Réalisation: Apprentissage automatique

```
Entrée [4]: def parse_file(file_in, file_out):
    fin = open(file_in)
    fout = io.open(file_out, "w", encoding="utf-8")
    lines = fin.readlines()
    res = []
    for i in range(len(lines)):
        line = lines[i].strip()
        if line.startswith("GET"):
            res.append("GET" + line.split(" ")[1])
        elif line.startswith("POST") or line.startswith("PUT"):
            url = line.split(' ')[0] + line.split(' ')[1]
            j = 1
            while True:
                if lines[i + j].startswith("Content-Length"):
                    break
                j += 1
            data = lines[i + j + 1].strip()
            url += '?' + data
            res.append(url)
    for line in res:
        line = urllib.parse.unquote(line).replace('\n', '').lower()
        fout.writelines(line + '\n')
    print("finished parse ", len(res), " requests")
    fout.close()
    fin.close()
```

```
Entrée [22]: # 3. N-grams (trigrams) Les plus fréquents dans Les requêtes
vectorizer = TfidfVectorizer(analyzer='char', ngram_range=(3, 3), max_features=20)
X_normal = vectorizer.fit_transform(good_requests)
X_anomalous = vectorizer.transform(bad_requests)

# Afficher Les n-grams Les plus fréquents
feature_names = np.array(vectorizer.get_feature_names_out())
normal_ngrams = np.asarray(X_normal.sum(axis=0)).flatten()
anomalous_ngrams = np.asarray(X_anomalous.sum(axis=0)).flatten()

# Trier Les n-grams par fréquence
normal_sorted_idx = normal_ngrams.argsort()[::-1]
anomalous_sorted_idx = anomalous_ngrams.argsort()[::-1]

# Afficher Les top 10 n-grams
print("Top 10 n-grams les plus fréquents dans les requêtes normales :)")
print(feature_names[normal_sorted_idx][:10])

print("Top 10 n-grams les plus fréquents dans les requêtes anormales :)")
print(feature_names[anomalous_sorted_idx][:10])
```

```
Top 10 n-grams les plus fréquents dans les requêtes normales :
['ost' 'cal' 'tie' '/lo' '/ti' 'nda' 'end' 'ien' 'oca' 'hos']
Top 10 n-grams les plus fréquents dans les requêtes anormales :
['ost' 'cal' 'oca' 'loc' '/lo' 'tp:' '080' '808' 'hos' 'alh']
```

```
# 3. N-grams (trigrams) Les plus fréquents dans Les requêtes
vectorizer = TfidfVectorizer(analyzer='char', ngram_range=(3, 3), max_features=100)
X_normal = vectorizer.fit_transform(good_requests)
X_anomalous = vectorizer.transform(bad_requests)

# Afficher Les n-grams Les plus fréquents
feature_names = np.array(vectorizer.get_feature_names_out())
normal_ngrams = np.asarray(X_normal.sum(axis=0)).flatten()
anomalous_ngrams = np.asarray(X_anomalous.sum(axis=0)).flatten()

# Trier Les n-grams par fréquence
normal_sorted_idx = normal_ngrams.argsort()[::-1]
anomalous_sorted_idx = anomalous_ngrams.argsort()[::-1]

# Afficher Les top 10 n-grams
print("Top 10 n-grams les plus fréquents dans les requêtes normales :)")
print(feature_names[normal_sorted_idx][:10])

print("Top 10 n-grams les plus fréquents dans les requêtes anormales :)")
print(feature_names[anomalous_sorted_idx][:10])
```

```
Top 10 n-grams les plus fréquents dans les requêtes normales :
['ost' 'get' 'eth' '/ti' 'tie' '/lo' 'cal' 'nda' 'end' 'ien']
Top 10 n-grams les plus fréquents dans les requêtes anormales :
['ost' 'ico' 'sp?' 'lic' 'ubl' 'bli' 'pub' '/pu' '1/p' 'co/']
```

N-grammes

```
Entrée [9]: vectorizer = TfidfVectorizer(min_df=0.0, analyzer="char", sublinear_tf=True, ngram_range=(3, 3))
X = vectorizer.fit_transform(all_requests)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=21)
```

Entrainement/Test

Réalisation:

Apprentissage automatique

```
Entrée [11]: #Decesion Tree
dtc = tree.DecisionTreeClassifier()
dtc.fit(X_train, y_train)
y_pred = dtc.predict(X_test)
score_test = metrics.accuracy_score(y_test, y_pred)
matrix = confusion_matrix(y_test, y_pred)
print ("Score Decesion Tree :",score_test)
print ("Confusion Matrix: ")
print (matrix)
```

```
Score Decesion Tree : 0.9921401670214508
Confusion Matrix:
[[3576  29]
 [ 19 2483]]
```

```
Entrée [13]: #Random Forest
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=200)
rfc.fit(X_train, y_train)
y_pred = rfc.predict(X_test)
score_test = metrics.accuracy_score(y_test, y_pred)
print ("Score Random Forest :",score_test)
print ("Confusion Matrix: ")
print (matrix)
```

```
Score Random Forest : 0.9862452922875389
Confusion Matrix:
[[3598   7]
 [   8 2494]]
```

```
Entrée [12]: #Linear SVM
linear_svm=LinearSVC(C=1)
linear_svm.fit(X_train, y_train)
y_pred = linear_svm.predict(X_test)
score_test = metrics.accuracy_score(y_test, y_pred)
matrix = confusion_matrix(y_test, y_pred)
print ("Score Linear SVM :",score_test)
print ("Confusion Matrix: ")
print (matrix)
```

```
Score Linear SVM : 0.9975438021942034
Confusion Matrix:
[[3598   7]
 [   8 2494]]
```

```
Entrée [10]: #Logistic regression
lgs = LogisticRegression()
lgs.fit(X_train, y_train)
y_pred = lgs.predict(X_test)
score_test = metrics.accuracy_score(y_test, y_pred)
matrix = confusion_matrix(y_test, y_pred)

print ("Score Logistic Regression :",score_test)
print ("Confusion Matrix: ")
print (matrix)
```

```
Score Logistic Regression : 0.9816603897167185
Confusion Matrix:
[[3558  47]
 [  65 2437]]
```


Réalisation:

Intégration

```
GNU nano 4.8                                predict.py
#!/usr/bin/env python3
import joblib
import sys
import os
import urllib.parse

# Charger le modèle et le vectorizer
model = joblib.load('svm_model.joblib')
vectorizer = joblib.load('vectorizer.joblib')

# Extraire la requête complète depuis les variables d'environnement
uri = os.environ.get('REQUEST_URI', '')
method = os.environ.get('REQUEST_METHOD', 'GET')

# Prétraitement
req_str = f"{method} {uri}"
req_str = urllib.parse.unquote(req_str)

# Prédiction
vect = vectorizer.transform([req_str])

# Log si anomalie
if pred[0] == 1:
    with open("/var/log/bunkerweb/ml_alerts.log", "a") as f:
        f.write(f"[ALERTE ML] Requête suspecte : {req_str}\n")

    # Optionnel : bloquer via sortie spécifique
    print("ModSecurity intervention -- ML detected anomaly.")
    sys.exit(1) # Peut être traité comme un blocage si intégré

sys.exit(0) # Pas de problème
```

```
Entrée [17]: import joblib
              joblib.dump(linear_svm, 'linear_svm_model.pkl')
```

```
Out[17]: ['linear_svm_model.pkl']
```

```
GNU nano 4.8                                ml_rules.conf
# 1) Si OWASP CRS n'a rien détecté (anomaly_score=0), on appelle le ML
SecRule TX:ANOMALY_SCORE "@eq 0" \
    "phase:2,id:999001,pass,nolog,ctl:ruleRemoveById=999002,exec:/usr/local/bin/ml_predict.py"

# 2) La règle qui bloque uniquement si ML détecte anomalie
SecRule TX:ML_ALERT "@eq 1" \
    "phase:2,id:999002,deny,status:403,log,msg:'Blocked by SVM prediction'"

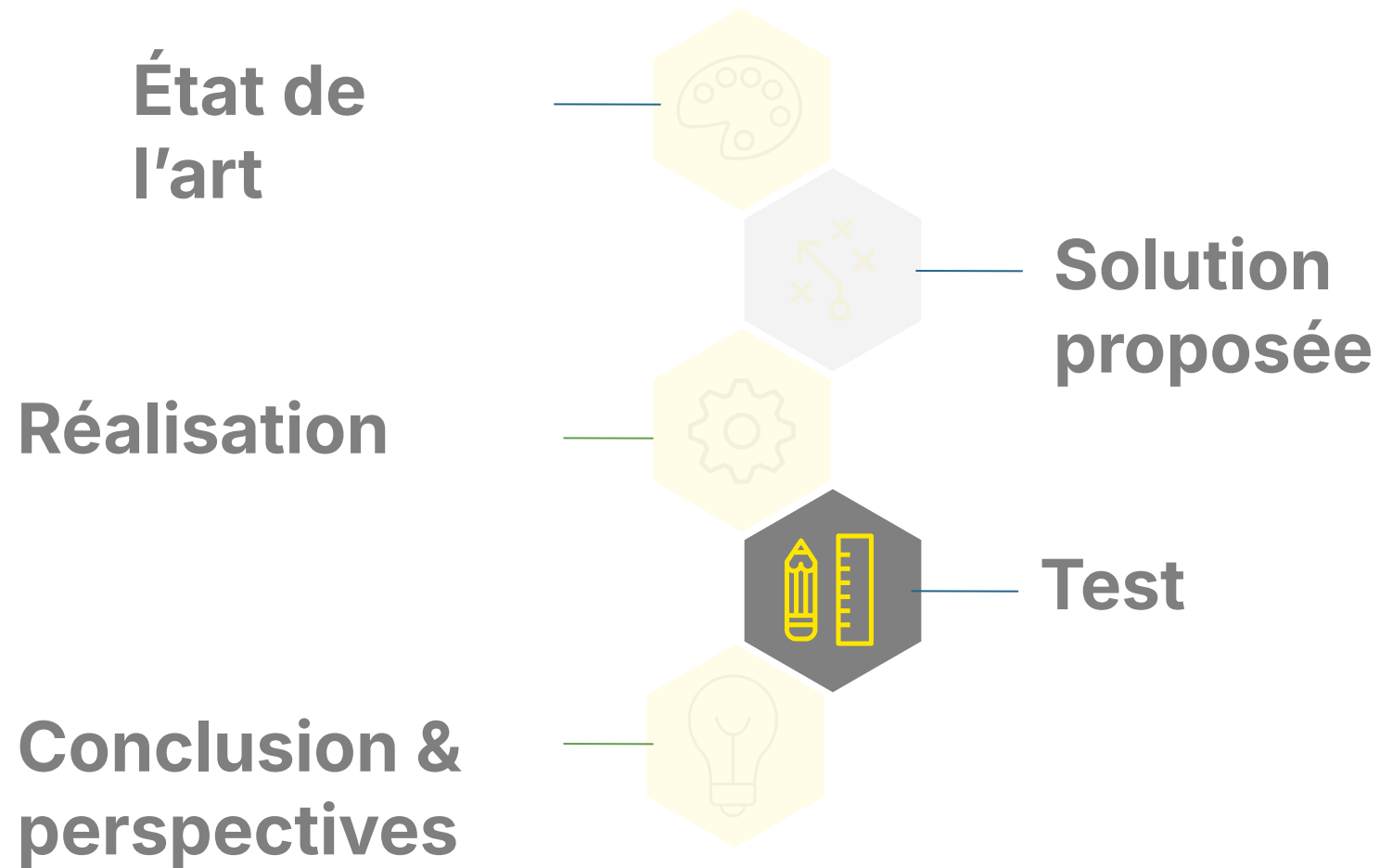
```

```
GNU nano 4.8                                modsecurity.conf-recommended
# -- Rule engine initialization -----

# Enable ModSecurity, attaching it to every transaction. Use detection
# only to start with, because that minimises the chances of post-installation
# disruption.
#
SecRuleEngine On

Include /etc/modsecurity/owasp-crs/crs-setup.conf
Include /etc/modsecurity/owasp-crs/rules/*.conf
Include /etc/modsecurity/ml_rules.conf
```


Plan



Test:

Environnement de test



Username

Password

Login



```
bw-scheduler: #gestionnaire des taches de bunkerweb
image: bunkerity/bunkerweb-scheduler:1.6.0
environment:
  <<: *bw-env
  BUNKERWEB_INSTANCES: "bunkerweb" # Définit l'instance de bunkerweb à gérer
  USE_REVERSE_PROXY: "yes"
  MULTISITE: "yes"
  SERVER_NAME: "dvwa.com"
  dvwa.com_SERVER_TYPE: "http"
  dvwa.com_USE_REVERSE_PROXY: "yes"
  dvwa.com_REVERSE_PROXY_HOST: "http://dvwa:80"
  dvwa.com_REVERSE_PROXY_URL: "/"
  UI_HOST: "http://bw-ui:7000" # Change it if needed
volumes:
  - bw-data:/data # This is used to persist the cache and other data like th>
restart: "unless-stopped"
```

OWASP/WAFEC



1 Contributor 0 Issues 0 Stars 2 Forks



Test:

Tests de fonctionnalités

PLEASE PROVE THAT YOU ARE HUMAN BEFORE ACCESSING THIS WEBSITE

Z f M C D R

Enter the text above

Submit

BUNKER
web

This website is protected with **BunkerWeb**

Activate blacklisting

☒ USE_BLACKLIST

?

Blacklist IP/network

?

BLACKLIST_IP
10.0.2.15

Blacklist reverse DNS

?

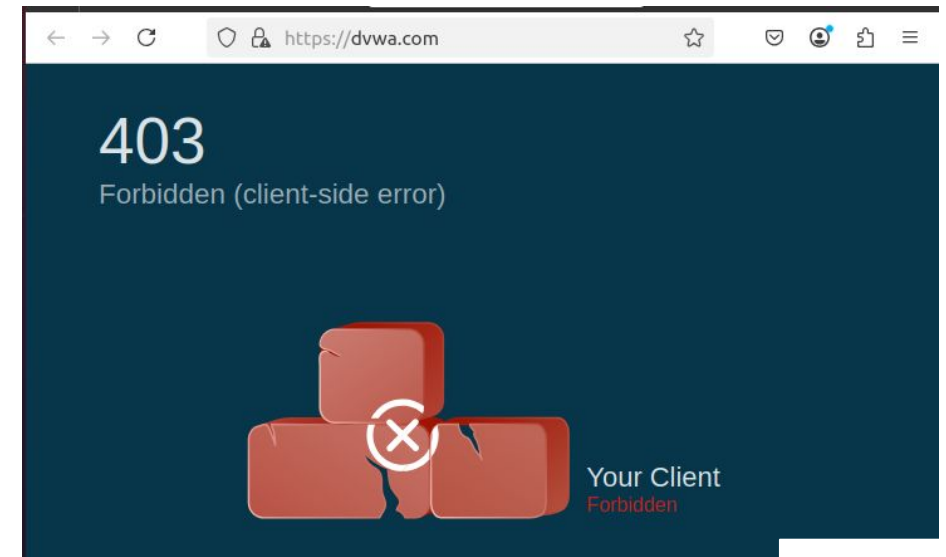
Blacklist reverse DNS global IPs

?

BLACKLIST_RDNS
.shodan.io .censys.io

☒ BLACKLIST_RDNS_GLOBAL

```
bunkerweb_1 | 2025/06/07 15:00:48 [error] 319#319: *347 [client 10.0.2.15]
ModSecurity: Access denied with code 403 (phase 2). Matched "Operator `Ge' with
parameter `5' against variable `TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: `5'
) [file "/usr/share/bunkerweb/core/modsecurity/files/coreruleset-v4/rules/REQU
EST-949-BLOCKING-EVALUATION.conf"] [line "222"] [id "949110"] [rev "" ] [msg ""
] [data "" ] [severity "0"] [ver "OWASP CRS/4.12.0"] [maturity "0"] [accuracy "0"
] [tag "anomaly-evaluation"] [tag "OWASP CRS"] [hostname "172.18.0.2"] [uri "/v
ulnerabilities/sqli/"] [unique_id "174930844840.464260"] [ref "" ], client: 10.0
.2.15, server: dvwa.com, request: "GET /vulnerabilities/sqli?id=may+%27OR+%271
%27%3D%271&Submit=Submit HTTP/2.0", host: "dvwa.com", referer: "https://dvwa.c
om/vulnerabilities/sqli/"
```



Test:

Tests automatisés

```
(may@kali)-[~/XSStrike]
$ python3 xsstrike.py -u "https://192.168.100.5/login.php?username=admin"
```

XSStrike

Checking for DOM vulnerabilities

```
[~] WAF detected: Amazon Web Services Web Application Firewall (Amazon)
[!] Testing parameter: username
[~] No reflection found
```

```
(may@kali)-[~/XSStrike]
$
```

```
(may@kali)-[~/XSStrike]
$
```

```
(may@kali)-[~/Desktop]
$ gobuster dir -u https://dvwa.com -w /usr/share/wordlists/dirb/common.txt -k -b 403
```

Gobuster v3.6

by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

```
[+] Url: https://dvwa.com
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 403
[+] User Agent: gobuster/3.6
[+] Timeout: 10s
```

Starting gobuster in directory enumeration mode

Progress: 4614 / 4615 (99.98%)

Finished

```
(may@kali)-[~/Desktop]
$
```

```
(may@kali)-[~/Desktop]
$ sqlmap -u "https://dvwa.com/login.php?id=1" --batch --random-agent
```

1.9#stable

<https://sqlmap.org>

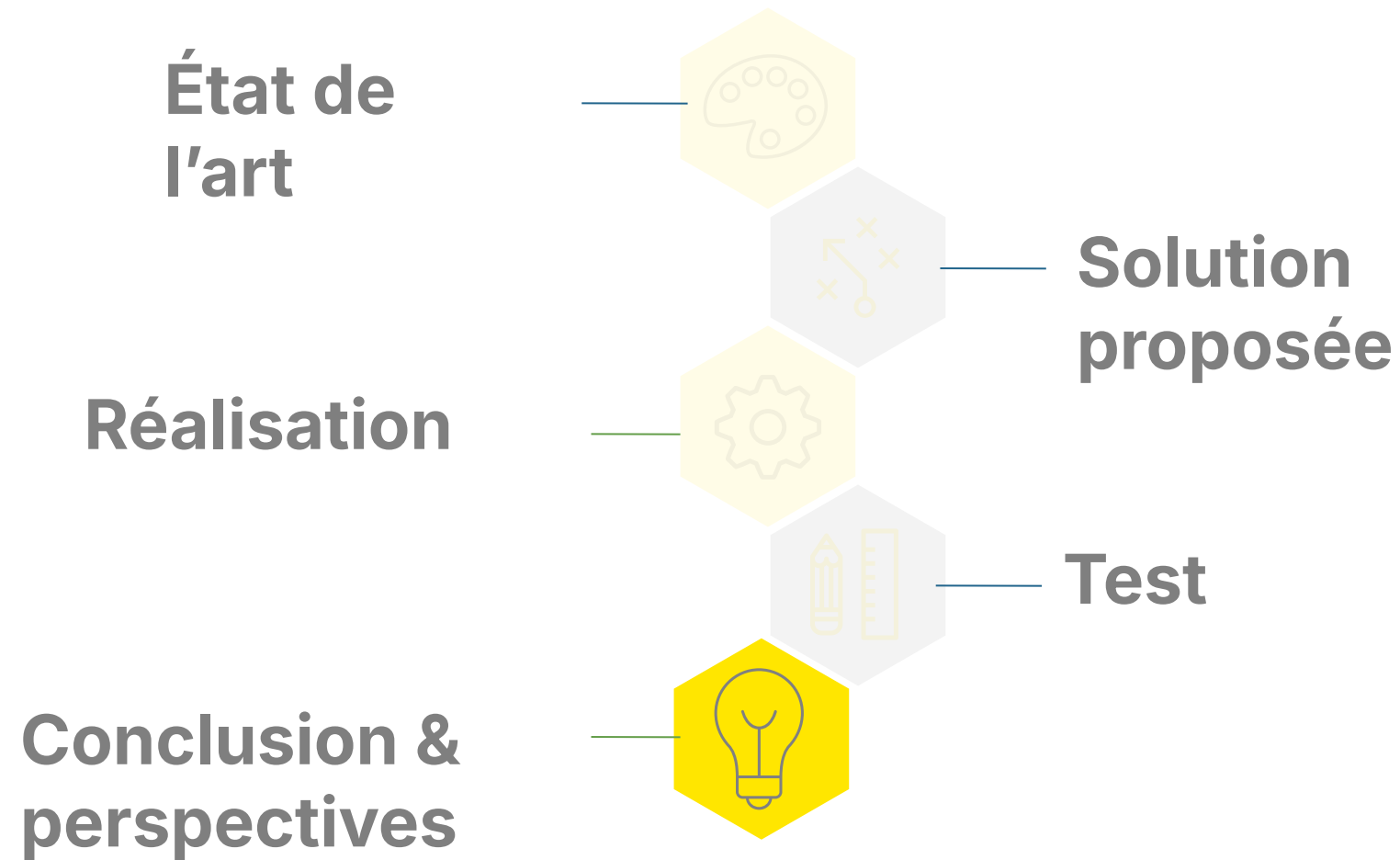
```
[21:56:38] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 68 times, 429 (Too Many Requests) - 4 times
```

```
[*] ending @ 21:56:38 /2025-04-16/
```

```
(may@kali)-[~/Desktop]
$
```

```
(may@kali)-[/usr/share/seclists/Fuzzing]
$ ffuf -X POST \
-u https://dvwa.com/login.php \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=FUZZ" \
-w /usr/share/seclists/Fuzzing/login_bypass.txt \
-rate 5
```

Plan



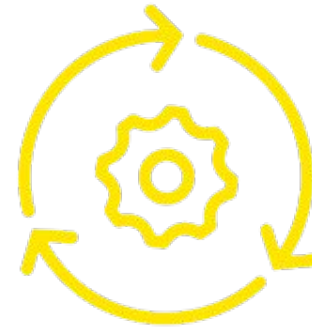
Conclusion



Différentes recherches ont été menées sur diverses solutions WAF.



Déploiement de BunkerWeb

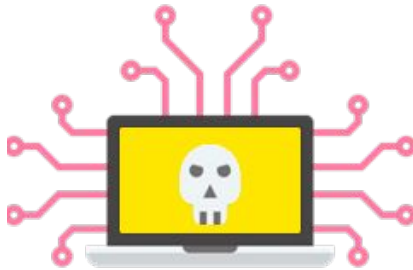


Renforcement par un module d'apprentissage automatique (SVM)

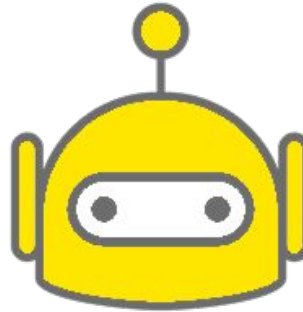


Evaluation avec des outils de pentesting

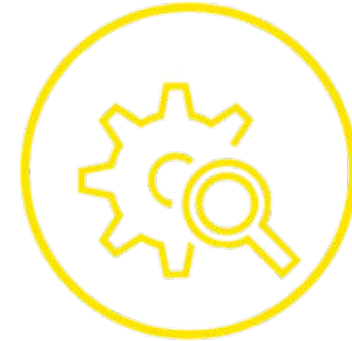
Perspectives



Structuration des
logs du modèle au
format JSON



Mise en place d'une
politique de sécurité
dynamique



Vers un WAF
autonome et
résilient